

# Graders

Learn about graders used for evals and fine-tuning.

Copy Page

Graders are a way to evaluate your model's performance against reference answers. Our [graders API](#) is a way to test your graders, experiment with results, and improve your fine-tuning or evaluation framework to get the results you want.

## Overview

Graders let you compare reference answers to the corresponding model-generated answer and return a grade in the range from 0 to 1. It's sometimes helpful to give the model partial credit for an answer, rather than a binary 0 or 1.

Graders are specified in JSON format, and there are several types:

- [String check](#)
- [Text similarity](#)
- [Score model grader](#)
- [Python code execution](#)

In reinforcement fine-tuning, you can nest and combine graders by using [multigraders](#).

Use this guide to learn about each grader type and see starter examples. To build a grader and get started with reinforcement fine-tuning, see the [RFT guide](#). Or to get started with evals, see the [Eval guide](#).

## Templating

The inputs to certain graders use a templating syntax to grade multiple examples with the same configuration. Any string with `{{ }}` double curly braces will be substituted with the variable value.

Each input inside the `{{ }}` must include a *namespace* and a *variable* with the following format `{{ namespace.variable }}`. The only supported namespaces are

`item` and `sample`

## Templating

The inputs to certain graders use a templating syntax to grade multiple examples with the same configuration. Any string with `{{ }}` double curly braces will be substituted with the variable value.

Each input inside the `{{}}` must include a *namespace* and a *variable* with the following format `{{ namespace.variable }}`. The only supported namespaces are `item` and `sample`.

All nested variables can be accessed with JSON path like syntax.

### Item namespace

The item namespace will be populated with variables from the input data source for evals, and from each dataset item for fine-tuning. For example, if a row contains the following

```
1 {  
2   "reference_answer": "..."  
3 }
```



This can be used within the grader as `{{ item.reference_answer }}`.

### Sample namespace

The sample namespace will be populated with variables from the model sampling step during evals or during the fine-tuning step. The following variables are included

- `output_text`, the model output content as a string.
- `output_json`, the model output content as a JSON object, only if `response_format` is included in the sample.
- `output_tools`, the model output `tool_calls`, which have the same structure as output tool calls in the [chat completions API](#).
- `choices`, the output choices, which has the same structure as output choices in the [chat completions API](#).
- `output_audio`, the model audio output object containing Base64-encoded `data` and a `transcript`.

For example, to access the model output content as a string, `{{ sample.output_text }}` can be used within the grader.

`choices`, the output choices, which has the same structure as output choices in the [chat completions API](#).

- `output_audio`, the model audio output object containing Base64-encoded `data` and a `transcript`.

For example, to access the model output content as a string, `{{ sample.output_text }}` can be used within the grader.

## > Details on grading tool calls

### String check grader

Use these simple string operations to return a 0 or 1. String check graders are good for scoring straightforward pass or fail answers—for example, the correct name of a city, a yes or no answer, or an answer containing or starting with the correct information.

```
1 {
2   "type": "string_check",
3   "name": string,
4   "operation": "eq" | "ne" | "like" | "ilike",
5   "input": string,
6   "reference": string,
7 }
```

Operations supported for string-check-grader are:

- `eq`: Returns 1 if the input matches the reference (case-sensitive), 0 otherwise
- `neq`: Returns 1 if the input does not match the reference (case-sensitive), 0 otherwise
- `like`: Returns 1 if the input contains the reference (case-sensitive), 0 otherwise
- `ilike`: Returns 1 if the input contains the reference (not case-sensitive), 0 otherwise

### Text similarity grader

Use text similarity graders when to evaluate how close the model-generated output is to the reference, scored with various evaluation frameworks.

This is useful for open-ended text responses. For example, if your dataset contains reference answers from experts in paragraph form, it's helpful to see how close your

otherwise

- `like` : Returns 1 if the input contains the reference (case-sensitive), 0 otherwise
- `ilike` : Returns 1 if the input contains the reference (not case-sensitive), 0 otherwise

## Text similarity grader

Use text similarity graders when to evaluate how close the model-generated output is to the reference, scored with various evaluation frameworks.

This is useful for open-ended text responses. For example, if your dataset contains reference answers from experts in paragraph form, it's helpful to see how close your model-generated answer is to that content, in numerical form.

```
1 {
2   "type": "text_similarity",
3   "name": string,
4   "input": string,
5   "reference": string,
6   "pass_threshold": number,
7   "evaluation_metric": "fuzzy_match" | "bleu" | "gleu" | "meteor" | "cosine"
8 }
```

Operations supported for `string-similarity-grader` are:

- `fuzzy_match` : Fuzzy string match between input and reference, using `rapidfuzz`
- `bleu` : Computes the BLEU score between input and reference
- `gleu` : Computes the Google BLEU score between input and reference
- `meteor` : Computes the METEOR score between input and reference
- `cosine` : Computes Cosine similarity between embedded input and reference, using `text-embedding-3-large`. Only available for evals.
- `rouge-*` : Computes the ROUGE score between input and reference

## Model graders

In general, using a model grader means prompting a separate model to grade the outputs of the model you're fine-tuning. Your two models work together to do reinforcement fine-tuning. The *grader model* evaluates the *training model*.

# OpenAI Developers



outputs of the model you're fine-tuning. Your two models work together to do reinforcement fine-tuning. The *grader model* evaluates the *training model*.

## Score model graders

A score model grader will take the input and return a numeric score based on the prompt within the given range.

```
1  {  
2      "type": "score_model",  
3      "name": string,  
4      "input": Message[],  
5      "model": string,  
6      "pass_threshold": number,  
7      "range": number[],  
8      "sampling_params": {  
9          "seed": number,  
10         "top_p": number,  
11         "temperature": number,  
12         "max_completions_tokens": number,  
13         "reasoning_effort": "minimal" | "low" | "medium" | "high"  
14     }  
15 }
```

Where each message is of the following form:

```
1  {  
2      "role": "system" | "developer" | "user" | "assistant",  
3      "content": str  
4  }
```

To use a score model grader, the input is a list of chat messages, each containing a `role` and `content`. The output of the grader will be truncated to the given `range`, and default to 0 for all non-numeric outputs. Within each message, the same templating can be used as with other common graders to reference the ground truth or model sample.

Here's a full runnable code sample:

```
1  import os  
2  import requests  
3  
4  # get the API key from environment
```

# OpenAI Developers



```
1  import os
2  import requests
3
4  # get the API key from environment
5  api_key = os.environ["OPENAI_API_KEY"]
6  headers = {"Authorization": f"Bearer {api_key}"}
7
8  # define a dummy grader for illustration purposes
9  grader = {
10     "type": "score_model",
11     "name": "my_score_model",
12     "input": [
13         {
14             "role": "system",
15             "content": "You are an expert grader. If the reference and model answers are the same, return 1.0. Otherwise, return 0.0."
16         },
17         {
18             "role": "user",
19             "content": "Reference: {{ item.reference_answer }}. Model answer: {{ item.model_answer }}"
20         }
21     ],
22     "pass_threshold": 0.5,
23     "model": "o4-mini-2025-04-16",
24     "range": [0, 1],
25     "sampling_params": {
26         "max_completions_tokens": 32768,
27         "top_p": 1,
28         "reasoning_effort": "medium"
29     },
30 }
31
32 # validate the grader
33 payload = {"grader": grader}
34 response = requests.post(
35     "https://api.openai.com/v1/fine_tuning/alpha/graders/validate",
36     json=payload,
37     headers=headers
38 )
39 print("validate response:", response.text)
40
41 # run the grader with a test reference and sample
42 payload = {
43     "grader": grader,
44     "item": {
45         "reference_answer": 1.0
46     },
47     "model_sample": "0.9"
48 }
49 response = requests.post(
```

```
46 },
47 "model_sample": "0.9"
48 }
49 response = requests.post(
50     "https://api.openai.com/v1/fine_tuning/alpha/graders/run",
51     json=payload,
52     headers=headers
53 )
54 print("run response:", response.text)
```

## Score model grader outputs

Under the hood, the `score_model` grader will query the requested model with the provided prompt and sampling parameters and will request a response in a specific response format. The response format that is used is provided below

```
1 {
2   "result": float,
3   "steps": ReasoningStep[],
4 }
```



Where each reasoning step is of the form

```
1 {
2   description: string,
3   conclusion: string
4 }
```



This format queries the model not just for the numeric `result` (the reward value for the query), but also provides the model some space to think through the reasoning behind the score. When you are writing your grader prompt, it may be useful to refer to these two fields by name explicitly (e.g. “include reasoning about the type of chemical bonds present in the molecule in the conclusion of your reasoning step”, or “return a value of -1.0 in the `result` field if the inputs do not satisfy condition X”).

## Model grader constraints

- Only the following models are supported for the `model` parameter
  - `gpt-4o-2024-08-06`
  - `gpt-4o-mini-2024-07-18`
  - `gpt-4.1-2025-04-14`

```

3     CONCLUSION: string
4 }

```

This format queries the model not just for the numeric `result` (the reward value for the query), but also provides the model some space to think through the reasoning behind the score. When you are writing your grader prompt, it may be useful to refer to these two fields by name explicitly (e.g. “include reasoning about the type of chemical bonds present in the molecule in the conclusion of your reasoning step”, or “return a value of -1.0 in the `result` field if the inputs do not satisfy condition X”).

## Model grader constraints

- Only the following models are supported for the `model` parameter`
  - `gpt-4o-2024-08-06`
  - `gpt-4o-mini-2024-07-18`
  - `gpt-4.1-2025-04-14`
  - `gpt-4.1-mini-2025-04-14`
  - `gpt-4.1-nano-2025-04-14`
  - `o1-2024-12-17`
  - `o3-mini-2025-01-31`
  - `o3-2025-04-16`
  - `o4-mini-2025-04-16`
- `temperature` changes not supported for reasoning models.
- `reasoning_effort` is not supported for non-reasoning models.

## How to write grader prompts

Writing grader prompts is an iterative process. The best way to iterate on a model grader prompt is to create a model grader eval. To do this, you need:

- Task prompts:** Write extremely detailed prompts for the desired task, with step-by-step instructions and many specific examples in context.
- Answers generated by a model or human expert:** Provide many high quality examples of answers, both from the model and trusted human experts.
- Corresponding ground truth grades for those answers:** Establish what a good grade looks like. For example, your human expert grades should be 1.

Then you can automatically evaluate how effectively the model grader distinguishes answers of different quality levels. Over time, add edge cases into your model grader



# OpenAI Developers



2. **Answers generated by a model or human expert:** Provide many high quality examples of answers, both from the model and trusted human experts.
3. **Corresponding ground truth grades for those answers:** Establish what a good grade looks like. For example, your human expert grades should be 1.

Then you can automatically evaluate how effectively the model grader distinguishes answers of different quality levels. Over time, add edge cases into your model grader eval as you discover and patch them with changes to the prompt.

For example, say you know from your human experts which answers are best:

```
answer_1 > answer_2 > answer_3
```



Verify that the model grader's answers match that:

```
model_grader(answer_1, reference_answer) > model_grader(answer_2, reference_answer)
```



## Grader hacking

Models being trained sometimes learn to exploit weaknesses in model graders, also known as “grader hacking” or “reward hacking.” You can detect this by checking the model’s performance across model grader evals and expert human evals. A model that’s hacked the grader will score highly on model grader evals but score poorly on expert human evaluations. Over time, we intend to improve observability in the API to make it easier to detect this during training.

## Python graders

This grader allows you to execute arbitrary python code to grade the model output. The grader expects a grade function to be present that takes in two arguments and outputs a float value. Any other result (exception, invalid float value, etc.) will be marked as invalid and return a 0 grade.

```
1 {  
2   "type": "python",  
3   "source": "def grade(sample, item):\n    return 1.0",  
4   "image_tag": "2025-05-08"  
5 }
```



# OpenAI Developers



marked as invalid and return a 0 grade.

```
1 {
2   "type": "python",
3   "source": "def grade(sample, item):\n    return 1.0",
4   "image_tag": "2025-05-08"
5 }
```

The python source code must contain a grade function that takes in exactly two arguments and returns a float value as a grade.

```
1 from typing import Any
2
3 def grade(sample: dict[str, Any], item: dict[str, Any]) -> float:
4     # your logic here
5     return 1.0
```

The first argument supplied to the grading function will be a dictionary populated with the model's output during training for you to grade. `output_json` will only be populated if the output uses `response_format`.

```
1 {
2   "choices": [...],
3   "output_text": "...",
4   "output_json": {},
5   "output_tools": [...],
6   "output_audio": {}
7 }
```

The second argument supplied is a dictionary populated with input grading context. For evals, this will include keys from the data source. For fine-tuning this will include keys from each training data row.

```
1 {
2   "reference_answer": "...",
3   "my_key": {...}
4 }
```

Here's a working example:

```
1 import os
```

# OpenAI Developers



Here's a working example:

```
1  import os
2  import requests
3
4  # get the API key from environment
5  api_key = os.environ["OPENAI_API_KEY"]
6  headers = {"Authorization": f"Bearer {api_key}"}
7
8  grading_function = """
9  from rapidfuzz import fuzz, utils
10
11 def grade(sample, item) -> float:
12     output_text = sample["output_text"]
13     reference_answer = item["reference_answer"]
14     return fuzz.WRatio(output_text, reference_answer, processor=utils.default_processor)
15 """
16
17 # define a dummy grader for illustration purposes
18 grader = {
19     "type": "python",
20     "source": grading_function
21 }
22
23 # validate the grader
24 payload = {"grader": grader}
25 response = requests.post(
26     "https://api.openai.com/v1/fine_tuning/alpha/graders/validate",
27     json=payload,
28     headers=headers
29 )
30 print("validate request_id:", response.headers["x-request-id"])
31 print("validate response:", response.text)
32
33 # run the grader with a test reference and sample
34 payload = {
35     "grader": grader,
36     "item": {
37         "reference_answer": "fuzzy wuzzy had no hair"
38     },
39     "model_sample": "fuzzy wuzzy was a bear"
40 }
41 response = requests.post(
42     "https://api.openai.com/v1/fine_tuning/alpha/graders/run",
43     json=payload,
44     headers=headers
45 )
46 print("run request_id:", response.headers["x-request-id"])
```

# OpenAI Developers



```
28     headers=headers
29 )
30 print("validate request_id:", response.headers["x-request-id"])
31 print("validate response:", response.text)
32
33 # run the grader with a test reference and sample
34 payload = {
35     "grader": grader,
36     "item": {
37         "reference_answer": "fuzzy wuzzy had no hair"
38     },
39     "model_sample": "fuzzy wuzzy was a bear"
40 }
41 response = requests.post(
42     "https://api.openai.com/v1/fine_tuning/alpha/graders/run",
43     json=payload,
44     headers=headers
45 )
46 print("run request_id:", response.headers["x-request-id"])
47 print("run response:", response.text)
```

**Tip:** If you don't want to manually put your grading function in a string, you can also load it from a Python file using `importlib` and `inspect`. For example, if your grader function is in a file named `grader.py`, you can do:

```
1 import importlib
2 import inspect
3
4 grader_module = importlib.import_module("grader")
5 grader = {
6     "type": "python",
7     "source": inspect.getsource(grader_module)
8 }
```



This will automatically use the entire source code of your `grader.py` file as the grader which can be helpful for longer graders.

## Technical constraints

- Your uploaded code must be less than `256kB` and will not have network access.
- The grading execution itself is limited to 2 minutes.
- At runtime you will be given a limit of 2Gb of memory and 1Gb of disk space to use.
- There's a limit of 2 CPU cores—any usage above this amount will result in

# OpenAI Developers



## throttling

The following third-party packages are available at execution time for the image tag

2025-05-08

```
1  numpy==2.2.4
2  scipy==1.15.2
3  sympy==1.13.3
4  pandas==2.2.3
5  rapidfuzz==3.10.1
6  scikit-learn==1.6.1
7  rouge-score==0.1.2
8  deepdiff==8.4.2
9  jsonschema==4.23.0
10 pydantic==2.10.6
11 pyyaml==6.0.2
12 nltk==3.9.1
13 sqlparse==0.5.3
14 rdkit==2024.9.6
15 scikit-bio==0.6.3
16 ast-grep-py==0.36.2
```



Additionally the following nltk corpora are available:

```
1  punkt
2  stopwords
3  wordnet
4  omw-1.4
5  names
```



## Multigraders

“Currently, this grader is only used for Reinforcement fine-tuning”

A `multigrader` object combines the output of multiple graders to produce a single score. Multigraders work by computing grades over the fields of other grader objects and turning those sub-grades into an overall grade. This is useful when a correct answer depends on multiple things being true—for example, that the text is similar *and* that the answer contains a specific string.

As an example, say you wanted the model to output JSON with the following two fields:

```
4 omw-1.4
5 names
```

## Multigraders

“Currently, this grader is only used for Reinforcement fine-tuning”

A `multigrader` object combines the output of multiple graders to produce a single score. Multigraders work by computing grades over the fields of other grader objects and turning those sub-grades into an overall grade. This is useful when a correct answer depends on multiple things being true—for example, that the text is similar *and* that the answer contains a specific string.

As an example, say you wanted the model to output JSON with the following two fields:

```
1 {
2   "name": "John Doe",
3   "email": "john.doe@gmail.com"
4 }
```

You’d want your grader to compare the two fields and then take the average between them.

You can do this by combining multiple graders into an object grader, and then defining a formula to calculate the output score based on each field:

```
1 {
2   "type": "multi",
3   "graders": {
4     "name": {
5       "name": "name_grader",
6       "type": "text_similarity",
7       "input": "{{sample.output_json.name}}",
8       "reference": "{{item.name}}",
9       "evaluation_metric": "fuzzy_match",
10      "pass_threshold": 0.9
11     },
12     "email": {
13       "name": "email_grader",
14       "type": "string_check",
15       "input": "{{sample.output_json.email}}",
16       "reference": "{{item.email}}",
```

# OpenAI Developers



```

7      "input": "{{sample.output_json.name}}",
8      "reference": "{{item.name}}",
9      "evaluation_metric": "fuzzy_match",
10     "pass_threshold": 0.9
11   },
12   "email": {
13     "name": "email_grader",
14     "type": "string_check",
15     "input": "{{sample.output_json.email}}",
16     "reference": "{{item.email}}",
17     "operation": "eq"
18   }
19 },
20 "calculate_output": "(name + email) / 2"
21 }

```

In this example, it's important for the model to get the email exactly right ( `string_check` returns either 0 or 1) but we tolerate some misspellings on the name ( `text_similarity` returns range from 0 to 1). Samples that get the email wrong will score between 0-0.5, and samples that get the email right will score between 0.5-1.0.

You cannot create a multigrader with a nested multigrader inside.

The calculate output field will have the keys of the input `graders` as possible variables and the following features are supported:

## Operators

- `+` (addition)
- `-` (subtraction)
- `*` (multiplication)
- `/` (division)
- `^` (power)

## Functions

- `min`
- `max`
- `abs`
- `floor`
- `ceil`
- `exp`
- `sqrt`

```
4 omw-1.4
5 names
```

## Multigraders

“Currently, this grader is only used for Reinforcement fine-tuning”

A `multigrader` object combines the output of multiple graders to produce a single score. Multigraders work by computing grades over the fields of other grader objects and turning those sub-grades into an overall grade. This is useful when a correct answer depends on multiple things being true—for example, that the text is similar *and* that the answer contains a specific string.

As an example, say you wanted the model to output JSON with the following two fields:

```
1 {
2   "name": "John Doe",
3   "email": "john.doe@gmail.com"
4 }
```



You’d want your grader to compare the two fields and then take the average between them.

You can do this by combining multiple graders into an object grader, and then defining a formula to calculate the output score based on each field:

```
1 {
2   "type": "multi",
3   "graders": {
4     "name": {
5       "name": "name_grader",
6       "type": "text_similarity",
7       "input": "{{sample.output_json.name}}",
8       "reference": "{{item.name}}",
9       "evaluation_metric": "fuzzy_match",
10      "pass_threshold": 0.9
11     },
12     "email": {
13       "name": "email_grader",
14       "type": "string_check",
15       "input": "{{sample.output_json.email}}",
16       "reference": "{{item.email}}",
```





## throttling

The following third-party packages are available at execution time for the image tag

2025-05-08

```
1  numpy==2.2.4
2  scipy==1.15.2
3  sympy==1.13.3
4  pandas==2.2.3
5  rapidfuzz==3.10.1
6  scikit-learn==1.6.1
7  rouge-score==0.1.2
8  deepdiff==8.4.2
9  jsonschema==4.23.0
10 pydantic==2.10.6
11 pyyaml==6.0.2
12 nltk==3.9.1
13 sqlparse==0.5.3
14 rdkit==2024.9.6
15 scikit-bio==0.6.3
16 ast-grep-py==0.36.2
```



Additionally the following nltk corpora are available:

```
1  punkt
2  stopwords
3  wordnet
4  omw-1.4
5  names
```



## Multigraders

“Currently, this grader is only used for Reinforcement fine-tuning”

A `multigrader` object combines the output of multiple graders to produce a single score. Multigraders work by computing grades over the fields of other grader objects and turning those sub-grades into an overall grade. This is useful when a correct answer depends on multiple things being true—for example, that the text is similar *and* that the answer contains a specific string.

As an example, say you wanted the model to output JSON with the following two fields:

# OpenAI Developers



```
28     headers=headers
29 )
30 print("validate request_id:", response.headers["x-request-id"])
31 print("validate response:", response.text)
32
33 # run the grader with a test reference and sample
34 payload = {
35     "grader": grader,
36     "item": {
37         "reference_answer": "fuzzy wuzzy had no hair"
38     },
39     "model_sample": "fuzzy wuzzy was a bear"
40 }
41 response = requests.post(
42     "https://api.openai.com/v1/fine_tuning/alpha/graders/run",
43     json=payload,
44     headers=headers
45 )
46 print("run request_id:", response.headers["x-request-id"])
47 print("run response:", response.text)
```

**Tip:** If you don't want to manually put your grading function in a string, you can also load it from a Python file using `importlib` and `inspect`. For example, if your grader function is in a file named `grader.py`, you can do:

```
1 import importlib
2 import inspect
3
4 grader_module = importlib.import_module("grader")
5 grader = {
6     "type": "python",
7     "source": inspect.getsource(grader_module)
8 }
```



This will automatically use the entire source code of your `grader.py` file as the grader which can be helpful for longer graders.

## Technical constraints

- Your uploaded code must be less than `256kB` and will not have network access.
- The grading execution itself is limited to 2 minutes.
- At runtime you will be given a limit of 2Gb of memory and 1Gb of disk space to use.
- There's a limit of 2 CPU cores—any usage above this amount will result in

# OpenAI Developers



Here's a working example:

```
1  import os
2  import requests
3
4  # get the API key from environment
5  api_key = os.environ["OPENAI_API_KEY"]
6  headers = {"Authorization": f"Bearer {api_key}"}
7
8  grading_function = """
9  from rapidfuzz import fuzz, utils
10
11 def grade(sample, item) -> float:
12     output_text = sample["output_text"]
13     reference_answer = item["reference_answer"]
14     return fuzz.WRatio(output_text, reference_answer, processor=utils.default_processor)
15 """
16
17 # define a dummy grader for illustration purposes
18 grader = {
19     "type": "python",
20     "source": grading_function
21 }
22
23 # validate the grader
24 payload = {"grader": grader}
25 response = requests.post(
26     "https://api.openai.com/v1/fine_tuning/alpha/graders/validate",
27     json=payload,
28     headers=headers
29 )
30 print("validate request_id:", response.headers["x-request-id"])
31 print("validate response:", response.text)
32
33 # run the grader with a test reference and sample
34 payload = {
35     "grader": grader,
36     "item": {
37         "reference_answer": "fuzzy wuzzy had no hair"
38     },
39     "model_sample": "fuzzy wuzzy was a bear"
40 }
41 response = requests.post(
42     "https://api.openai.com/v1/fine_tuning/alpha/graders/run",
43     json=payload,
44     headers=headers
45 )
46 print("run request_id:", response.headers["x-request-id"])
```

# OpenAI Developers



marked as invalid and return a 0 grade.

```
1 {  
2   "type": "python",  
3   "source": "def grade(sample, item):\n    return 1.0",  
4   "image_tag": "2025-05-08"  
5 }
```

The python source code must contain a grade function that takes in exactly two arguments and returns a float value as a grade.

```
1 from typing import Any  
2  
3 def grade(sample: dict[str, Any], item: dict[str, Any]) -> float:  
4     # your logic here  
5     return 1.0
```

The first argument supplied to the grading function will be a dictionary populated with the model's output during training for you to grade. `output_json` will only be populated if the output uses `response_format`.

```
1 {  
2   "choices": [...],  
3   "output_text": "...",  
4   "output_json": {},  
5   "output_tools": [...],  
6   "output_audio": {}  
7 }
```

The second argument supplied is a dictionary populated with input grading context. For evals, this will include keys from the data source. For fine-tuning this will include keys from each training data row.

```
1 {  
2   "reference_answer": "...",  
3   "my_key": {...}  
4 }
```

Here's a working example:

```
1 import os
```

# OpenAI Developers



2. **Answers generated by a model or human expert:** Provide many high quality examples of answers, both from the model and trusted human experts.
3. **Corresponding ground truth grades for those answers:** Establish what a good grade looks like. For example, your human expert grades should be 1.

Then you can automatically evaluate how effectively the model grader distinguishes answers of different quality levels. Over time, add edge cases into your model grader eval as you discover and patch them with changes to the prompt.

For example, say you know from your human experts which answers are best:

```
answer_1 > answer_2 > answer_3
```



Verify that the model grader's answers match that:

```
model_grader(answer_1, reference_answer) > model_grader(answer_2, reference_answer)
```



## Grader hacking

Models being trained sometimes learn to exploit weaknesses in model graders, also known as “grader hacking” or “reward hacking.” You can detect this by checking the model’s performance across model grader evals and expert human evals. A model that’s hacked the grader will score highly on model grader evals but score poorly on expert human evaluations. Over time, we intend to improve observability in the API to make it easier to detect this during training.

## Python graders

This grader allows you to execute arbitrary python code to grade the model output. The grader expects a grade function to be present that takes in two arguments and outputs a float value. Any other result (exception, invalid float value, etc.) will be marked as invalid and return a 0 grade.

```
1 {  
2   "type": "python",  
3   "source": "def grade(sample, item):\n    return 1.0",  
4   "image_tag": "2025-05-08"  
5 }
```



```

3     CONCLUSION: string
4 }

```

This format queries the model not just for the numeric `result` (the reward value for the query), but also provides the model some space to think through the reasoning behind the score. When you are writing your grader prompt, it may be useful to refer to these two fields by name explicitly (e.g. “include reasoning about the type of chemical bonds present in the molecule in the conclusion of your reasoning step”, or “return a value of -1.0 in the `result` field if the inputs do not satisfy condition X”).

## Model grader constraints

- Only the following models are supported for the `model` parameter`
  - `gpt-4o-2024-08-06`
  - `gpt-4o-mini-2024-07-18`
  - `gpt-4.1-2025-04-14`
  - `gpt-4.1-mini-2025-04-14`
  - `gpt-4.1-nano-2025-04-14`
  - `o1-2024-12-17`
  - `o3-mini-2025-01-31`
  - `o3-2025-04-16`
  - `o4-mini-2025-04-16`
- `temperature` changes not supported for reasoning models.
- `reasoning_effort` is not supported for non-reasoning models.

## How to write grader prompts

Writing grader prompts is an iterative process. The best way to iterate on a model grader prompt is to create a model grader eval. To do this, you need:

- Task prompts:** Write extremely detailed prompts for the desired task, with step-by-step instructions and many specific examples in context.
- Answers generated by a model or human expert:** Provide many high quality examples of answers, both from the model and trusted human experts.
- Corresponding ground truth grades for those answers:** Establish what a good grade looks like. For example, your human expert grades should be 1.

Then you can automatically evaluate how effectively the model grader distinguishes answers of different quality levels. Over time, add edge cases into your model grader

```
46 },
47 "model_sample": "0.9"
48 }
49 response = requests.post(
50     "https://api.openai.com/v1/fine_tuning/alpha/graders/run",
51     json=payload,
52     headers=headers
53 )
54 print("run response:", response.text)
```

## Score model grader outputs

Under the hood, the `score_model` grader will query the requested model with the provided prompt and sampling parameters and will request a response in a specific response format. The response format that is used is provided below

```
1 {
2   "result": float,
3   "steps": ReasoningStep[],
4 }
```



Where each reasoning step is of the form

```
1 {
2   description: string,
3   conclusion: string
4 }
```



This format queries the model not just for the numeric `result` (the reward value for the query), but also provides the model some space to think through the reasoning behind the score. When you are writing your grader prompt, it may be useful to refer to these two fields by name explicitly (e.g. “include reasoning about the type of chemical bonds present in the molecule in the conclusion of your reasoning step”, or “return a value of -1.0 in the `result` field if the inputs do not satisfy condition X”).

## Model grader constraints

- Only the following models are supported for the `model` parameter
  - `gpt-4o-2024-08-06`
  - `gpt-4o-mini-2024-07-18`
  - `gpt-4.1-2025-04-14`

# OpenAI Developers



```
1  import os
2  import requests
3
4  # get the API key from environment
5  api_key = os.environ["OPENAI_API_KEY"]
6  headers = {"Authorization": f"Bearer {api_key}"}
7
8  # define a dummy grader for illustration purposes
9  grader = {
10     "type": "score_model",
11     "name": "my_score_model",
12     "input": [
13         {
14             "role": "system",
15             "content": "You are an expert grader. If the reference and model answers are the same, return 1.0. Otherwise, return 0.0."
16         },
17         {
18             "role": "user",
19             "content": "Reference: {{ item.reference_answer }}. Model answer: {{ item.model_answer }}"
20         }
21     ],
22     "pass_threshold": 0.5,
23     "model": "o4-mini-2025-04-16",
24     "range": [0, 1],
25     "sampling_params": {
26         "max_completions_tokens": 32768,
27         "top_p": 1,
28         "reasoning_effort": "medium"
29     },
30 }
31
32 # validate the grader
33 payload = {"grader": grader}
34 response = requests.post(
35     "https://api.openai.com/v1/fine_tuning/alpha/graders/validate",
36     json=payload,
37     headers=headers
38 )
39 print("validate response:", response.text)
40
41 # run the grader with a test reference and sample
42 payload = {
43     "grader": grader,
44     "item": {
45         "reference_answer": 1.0
46     },
47     "model_sample": "0.9"
48 }
49 response = requests.post(
```



# OpenAI Developers



outputs of the model you're fine-tuning. Your two models work together to do reinforcement fine-tuning. The *grader model* evaluates the *training model*.

## Score model graders

A score model grader will take the input and return a numeric score based on the prompt within the given range.

```
1  {  
2      "type": "score_model",  
3      "name": string,  
4      "input": Message[],  
5      "model": string,  
6      "pass_threshold": number,  
7      "range": number[],  
8      "sampling_params": {  
9          "seed": number,  
10         "top_p": number,  
11         "temperature": number,  
12         "max_completions_tokens": number,  
13         "reasoning_effort": "minimal" | "low" | "medium" | "high"  
14     }  
15 }
```

Where each message is of the following form:

```
1  {  
2      "role": "system" | "developer" | "user" | "assistant",  
3      "content": str  
4  }
```

To use a score model grader, the input is a list of chat messages, each containing a `role` and `content`. The output of the grader will be truncated to the given `range`, and default to 0 for all non-numeric outputs. Within each message, the same templating can be used as with other common graders to reference the ground truth or model sample.

Here's a full runnable code sample:

```
1  import os  
2  import requests  
3  
4  # get the API key from environment
```

otherwise

- `like` : Returns 1 if the input contains the reference (case-sensitive), 0 otherwise
- `ilike` : Returns 1 if the input contains the reference (not case-sensitive), 0 otherwise

## Text similarity grader

Use text similarity graders when to evaluate how close the model-generated output is to the reference, scored with various evaluation frameworks.

This is useful for open-ended text responses. For example, if your dataset contains reference answers from experts in paragraph form, it's helpful to see how close your model-generated answer is to that content, in numerical form.

```
1 {
2   "type": "text_similarity",
3   "name": string,
4   "input": string,
5   "reference": string,
6   "pass_threshold": number,
7   "evaluation_metric": "fuzzy_match" | "bleu" | "gleu" | "meteor" | "cosine"
8 }
```

Operations supported for `string-similarity-grader` are:

- `fuzzy_match` : Fuzzy string match between input and reference, using `rapidfuzz`
- `bleu` : Computes the BLEU score between input and reference
- `gleu` : Computes the Google BLEU score between input and reference
- `meteor` : Computes the METEOR score between input and reference
- `cosine` : Computes Cosine similarity between embedded input and reference, using `text-embedding-3-large`. Only available for evals.
- `rouge-*` : Computes the ROUGE score between input and reference

## Model graders

In general, using a model grader means prompting a separate model to grade the outputs of the model you're fine-tuning. Your two models work together to do reinforcement fine-tuning. The *grader model* evaluates the *training model*.

# OpenAI Developers



`choices`, the output choices, which has the same structure as output choices in the [chat completions API](#).

- `output_audio`, the model audio output object containing Base64-encoded `data` and a `transcript`.

For example, to access the model output content as a string, `{{ sample.output_text }}` can be used within the grader.

## > Details on grading tool calls

### String check grader

Use these simple string operations to return a 0 or 1. String check graders are good for scoring straightforward pass or fail answers—for example, the correct name of a city, a yes or no answer, or an answer containing or starting with the correct information.

```
1 {
2   "type": "string_check",
3   "name": string,
4   "operation": "eq" | "ne" | "like" | "ilike",
5   "input": string,
6   "reference": string,
7 }
```



Operations supported for string-check-grader are:

- `eq`: Returns 1 if the input matches the reference (case-sensitive), 0 otherwise
- `neq`: Returns 1 if the input does not match the reference (case-sensitive), 0 otherwise
- `like`: Returns 1 if the input contains the reference (case-sensitive), 0 otherwise
- `ilike`: Returns 1 if the input contains the reference (not case-sensitive), 0 otherwise

### Text similarity grader

Use text similarity graders when to evaluate how close the model-generated output is to the reference, scored with various evaluation frameworks.

This is useful for open-ended text responses. For example, if your dataset contains reference answers from experts in paragraph form, it's helpful to see how close your

## Templating

The inputs to certain graders use a templating syntax to grade multiple examples with the same configuration. Any string with `{{ }}` double curly braces will be substituted with the variable value.

Each input inside the `{{}}` must include a *namespace* and a *variable* with the following format `{{ namespace.variable }}`. The only supported namespaces are `item` and `sample`.

All nested variables can be accessed with JSON path like syntax.

### Item namespace

The item namespace will be populated with variables from the input data source for evals, and from each dataset item for fine-tuning. For example, if a row contains the following

```
1 {  
2   "reference_answer": "..."  
3 }
```



This can be used within the grader as `{{ item.reference_answer }}`.

### Sample namespace

The sample namespace will be populated with variables from the model sampling step during evals or during the fine-tuning step. The following variables are included

- `output_text`, the model output content as a string.
- `output_json`, the model output content as a JSON object, only if `response_format` is included in the sample.
- `output_tools`, the model output `tool_calls`, which have the same structure as output tool calls in the [chat completions API](#).
- `choices`, the output choices, which has the same structure as output choices in the [chat completions API](#).
- `output_audio`, the model audio output object containing Base64-encoded `data` and a `transcript`.

For example, to access the model output content as a string, `{{ sample.output_text }}` can be used within the grader.

# Graders

Learn about graders used for evals and fine-tuning.

 Copy Page

Graders are a way to evaluate your model's performance against reference answers. Our [graders API](#) is a way to test your graders, experiment with results, and improve your fine-tuning or evaluation framework to get the results you want.

## Overview

Graders let you compare reference answers to the corresponding model-generated answer and return a grade in the range from 0 to 1. It's sometimes helpful to give the model partial credit for an answer, rather than a binary 0 or 1.

Graders are specified in JSON format, and there are several types:

- [String check](#)
- [Text similarity](#)
- [Score model grader](#)
- [Python code execution](#)

In reinforcement fine-tuning, you can nest and combine graders by using [multigraders](#).

Use this guide to learn about each grader type and see starter examples. To build a grader and get started with reinforcement fine-tuning, see the [RFT guide](#). Or to get started with evals, see the [Evals guide](#).

## Templating

The inputs to certain graders use a templating syntax to grade multiple examples with the same configuration. Any string with `{{ }}` double curly braces will be substituted with the variable value.

Each input inside the `{{ }}` must include a *namespace* and a *variable* with the following format `{{ namespace.variable }}`. The only supported namespaces are

`item` and `sample`

# OpenAI Developers



```

7      "input": "{{sample.output_json.name}}",
8      "reference": "{{item.name}}",
9      "evaluation_metric": "fuzzy_match",
10     "pass_threshold": 0.9
11   },
12   "email": {
13     "name": "email_grader",
14     "type": "string_check",
15     "input": "{{sample.output_json.email}}",
16     "reference": "{{item.email}}",
17     "operation": "eq"
18   }
19 },
20 "calculate_output": "(name + email) / 2"
21 }

```

In this example, it's important for the model to get the email exactly right ( `string_check` returns either 0 or 1) but we tolerate some misspellings on the name ( `text_similarity` returns range from 0 to 1). Samples that get the email wrong will score between 0-0.5, and samples that get the email right will score between 0.5-1.0.

You cannot create a multigrader with a nested multigrader inside.

The calculate output field will have the keys of the input `graders` as possible variables and the following features are supported:

## Operators

- `+` (addition)
- `-` (subtraction)
- `*` (multiplication)
- `/` (division)
- `^` (power)

## Functions

- `min`
- `max`
- `abs`
- `floor`
- `ceil`
- `exp`
- `sqrt`